

Unitree Week 23+24

Presentation by Mario Ortiz

This Week

- More work on Unitree R1 Reinforcement Learning code
- More work on Inverse Kinematics Code

RL Walking Improvements

- Previously, it was observed the movement of the R1 in the `rough_env_cfg.py` code showed that it slightly hopped, and there was a lack of usage of the knees when walking.
- This was fixed by adding custom rewards/penalties
 - Reward term “`both_feet_airborne`” utilizes a function to penalize the R1 when both feet are not touching the ground.
 - Function “`swing_knee_flexion_shortfall`” measures how far the corresponding knee moved from its default position.
 - The robot then gets penalized when the walking leg does not bend its knee by a specific threshold.

RL Walking Improvements (pt. 2)

- Decreased the desired foot air time threshold due to the new walking logic.
 - Discouraging excessively long swing phases.
- Increasing the penalty for hip roll to mitigate more leg-spreading motion.

RL Walking Logic

1. Receive forward and yaw commands →
2. Keep legs near default lateral position →
3. Maintain one or both feet in contact with the ground →
4. Lift the other foot for a short swing phase →
5. Bend the airborne leg's knee →
6. Place the swing foot down →
7. Alternate legs while tracking velocity.



```
#####  
Learning iteration 6999/7000  
  
Total steps: 172032000  
Steps per second: 8123  
Collection time: 2.801s  
Learning time: 0.224s  
Mean value loss: 0.0281  
Mean surrogate loss: -0.0049  
Mean entropy loss: 22.6933  
Mean reward: 13.53  
Mean episode length: 938.48  
Mean action std: 0.92  
Episode_Reward/track_lin_vel_xy_exp: 0.7876  
Episode_Reward/track_ang_vel_z_exp: 0.8070  
Episode_Reward/lin_vel_z_l2: 0.0000  
Episode_Reward/ang_vel_xy_l2: -0.0618  
Episode_Reward/dof_torques_l2: -0.0010  
Episode_Reward/dof_acc_l2: -0.0096  
Episode_Reward/action_rate_l2: -0.2912  
Episode_Reward/feet_air_time: 0.0119  
Episode_Reward/flat_orientation_l2: -0.0088  
Episode_Reward/dof_pos_limits: -0.0531  
Episode_Reward/termination_penalty: -0.0167  
Episode_Reward/both_feet_airborne: -0.0659  
Episode_Reward/swing_knee_flexion: -0.0005  
Episode_Reward/feet_slide: -0.0262  
Episode_Reward/joint_deviation_hip_roll: -0.0814  
Episode_Reward/joint_deviation_hip_yaw: -0.0226  
Episode_Reward/joint_deviation_waist: -0.0152  
Episode_Reward/joint_deviation_arms: -0.2053  
Episode_Reward/joint_deviation_head: -0.0456  
Curriculum/terrain_levels: 5.0938  
Metrics/base_velocity/error_vel_xy: 0.4259  
Metrics/base_velocity/error_vel_yaw: 1.2646  
Episode_Termination/time_out: 0.8872  
Episode_Termination/base_contact: 0.1128  
-----  
Iteration time: 3.03s  
Time elapsed: 05:56:45  
ETA: 00:00:00
```

Observations

- Improvements
 - Knees are used more for walking
 - Robots walk and ensure one foot is touching the ground while walking.
- Disadvantages
 - Does not change knee positions; it learns to walk one specific way that works for the entire simulation and sticks with it.

Inverse Kinematics Algorithm

- Fixed coding pipeline so that we can run the `prepare_amass_targets.py` and `retarget_amass_r1.py` by inputting any file (.npz input file containing human motion), initial frame, and number of frames made.
- One pipeline: run one code and it runs `prepare_amass_targets.py` and `retarget_amass_r1.py` based on specified inputs (allows for simplicity).
-

Terminal Output

```
PS C:\Users\Mario\IsaacLab> python scripts\r1\r1_IK_pipeline.py `
>> --amass-path "C:\Users\Mario\Documents\Homework\Internship
Stuff\YiSunInternship\CMU\CMU\01\01_01_poses.npz" `
>> --start-frame 0 `
>> --num-frames 1200
```

```
=====
=====
```

R1 AMASS Retargeting Pipeline

```
=====
=====
```

Source AMASS file:

C:\Users\Mario\Documents\Homework\Internship

Fixed Needed

hip_side_angle_deg mean=92.8436 max=96.9453

- This indicates a significant error with the hip_side_angle, meaning the logic for retargetting/fitting the R1 is incorrect.
 - The function “estimate_root_yaw_from_pelvis” computes the hip_side_angle_deg.
 - In the retarget_amass_r1.py, we had assumed the AMASS target yaw and the R1’s USD root yaw used the same axis convention → conversion was needed to fix this issue.

Terminal Output Fixed

```
PS C:\Users\Mario\IsaacLab> python scripts\r1\r1_IK_pipeline.py `
>> --amass-path "C:\Users\Mario\Documents\Homework\Internship
Stuff\YiSunInternship\CMU\CMU\01\01_01_poses.npz" `
>> --start-frame 0 `
>> --num-frames 1200 `
>> --name corrected_yaw
```

```
=====
=====
R1 AMASS Retargeting Pipeline
=====
=====
```

Source AMASS file:

AMASS Code Needed

Run Entire Pipeline (Targeting + Retargeting):

```
python scripts\r1\r1_IK_pipeline.py `
    --amass-path "C:\Users\Mario\Documents\Homework\Internship Stuff\YiSunInternship\CMU\CMU\01\01_01_poses.npz" ~
    -- start-frame {Specified Starting Frame} `
    -- num-frames {Number of Frames after starting frame} `
    -- name {Insert Desired Name of File}
```

Run Only Retargeting:

```
.\isaacclab.bat -p scripts\r1\retarget_amass_r1.py `
    --target-path "C:\Users\Mario\IsaacLab\scripts\r1\outputs\01_01_poses_f0_n1200_r1_targets.npz" `
    --output-path "C:\Users\Mario\IsaacLab\scripts\r1\outputs\01_01_poses_f0_n1200_r1_auto_yaw.npz"
```

Run Play Script:

```
.\isaacclab.bat -p scripts\r1\play_r1_motion.py `
    --motion-path "C:\Users\Mario\IsaacLab\scripts\r1\outputs\01_01_poses_f0_n1200_r1_auto_yaw.npz"
```

Other Interesting CMU Files

"\CMU\CMU\02\02_03_poses.npz" → Running Motion
"\CMU\CMU\02\02_04_poses.npz" → High Jump Motion
"\CMU\CMU\02\02_05_poses.npz" → Punch Motion
"\CMU\CMU\02\02_06_poses.npz" → Pick Up Item Motion?
"\CMU\CMU\05\05_02_poses.npz" → Dance
"\CMU\CMU\05\05_06_poses.npz" →

test_amass.py

- Inputs the AMASS .npz files as well as the smplh skeleton files and plots the joint movements on matplotlib.
- We can compare the initial joint movements with the retargeted R1 .npz file.

02_03_poses.npz

```
python .\scripts\r1\test_amass.py `
>> --amass-path
"C:\Users\Mario\Documents\Homework\Internship
Stuff\YiSunInternship\CMU\CMU\02\02_03_poses.npz"

.\isaacclab.bat -p scripts\r1\play_r1_motion.py --motion-path
"C:\Users\Mario\IsaacLab\scripts\r1\outputs\02_03_poses_172_f0_
n172_r1_retargeted.npz"
```

02_04_poses.npz

```
python .\scripts\r1\test_amass.py `  
>> --amass-path "C:\Users\Mario\Documents\Homework\Internship  
Stuff\YiSunInternship\CMU\CMU\02\02_04_poses.npz"  
  
.\isaacrab.bat -p scripts\r1\play_r1_motion.py --motion-path  
"C:\Users\Mario\IsaacLab\scripts\r1\outputs\02_04_poses_482_f0_n482_r1  
_retargeted.npz"
```

02_06_poses.npz

```
python .\scripts\r1\test_amass.py `  
>> --amass-path "C:\Users\Mario\Documents\Homework\Internship  
Stuff\YiSunInternship\CMU\CMU\02\02_06_poses.npz"  
  
.\isaacrab.bat -p scripts\r1\play_r1_motion.py --motion-path  
"C:\Users\Mario\IsaacLab\scripts\r1\outputs\02_06_poses_2234_f0_n2234  
_r1_retargeted.npz"
```

Reference-motion tracking with RL

- **Reference-motion tracking:** The policy gets rewarded for matching the retargeted motions.
- We can use the retargeted motion files for training.
 - Each retargeted file contains reference trajectory, which provides information on where the R1 should be at each frame (root_pos, root_quat, root_yaw, joint_pos, joint_names, fps/dt metadata, foot contacts/target metadata)
 - We use the reference trajectory to develop policies that output actions for training (such as desired joint positions).

Reference-Motion with RL vs. Pure Imitation Learning

- Imitation learning replicates expert behaviors directly from data, often without knowing the environment's goals.
 - Learns under physical control actions needed for balancing (AMASS does not provide such information).
- Reference Motion Tracking with RL trains an agent to mimic specific reference clips by penalizing the distance between the robot's pose and the reference frame.
 - ***Trains with RL to physically track reference trajectories.
 - **This is more effective initially as we currently have the kinematics of desired motions (.npz retarget files) which we can use as reference.**

Reference-Motion Tracking Steps

1. Obtaining several good retargeted R1 motions
2. Set up a motion dataset loader.
3. Set Up an R1 motion-tracking environment
 - a. Add rewards for joints, feet, root, body keypoints, etc.
 - b. Add random reference-frame resets.
4. Training Protocol using **PPO-RL** (similar to IsaacLab's provided locomotion policies)
5. After successful trainings, develop a curriculum of several successful trainings.